Mini-Projects: Introduction to Delphi tools

These interactive activities naturally build on each other, but can also be done in isolation if you only have a few minutes to stop by. Let the host know if you're interested in a particular section or topic! They can set you up with all precursor code.

Follow-along with the ⭐ items, or choose your own path!

🛈 symbols offer questions to help you get to know the data or system better.

⚔️ symbols provide "Side Quests" for exploring additional topics or diving deeper into a particular function

Access this worksheet later!

MINI-PROJECT 2: EDA and correlation analysis

The following activities can be done in R.

The R package `epiprocess` provides tools to manage, analyze, and process epidemiological data. `epiprocess` documentation and vignettes are available at cmu-delphi.github.io/epiprocess

2.0 Load the required packages

The `{InsightNetApr26}` package ensures all required Delphi tools and their correct versions/branches are installed for the purpose of this session. You can run the code below to install it.

```
if (!requireNamespace("pak", quietly = TRUE)) {
      install.packages("pak")
}
```

```
if (!requireNamespace("InsightNetApr26", quietly = TRUE)) {
     pak::pkg_install("cmu-delphi/InsightNet-apr-2026/InsightNetApr26")
}

InsightNetApr26::verify_setup()

library(epidatr)
library(epiprocess)
library(dplyr)
library(ggplot2)
```

2.1 General Exploratory Data Analysis (EDA) utilities

Explore indicators using `epiprocess`.

⭐ **Either continue with your data from Mini-Project 1 (in `epi_df` format) OR select a new data indicator of your choice and transform it to an `epi_df` using `epiprocess`. Make sure you pull the data for all states!**

For example:

```
epidata_nhsn_state <- pub_covidcast(
  source = "nhsn",
  signals = "confirmed_admissions_rsv_ew",
  geo_type = "state",
  time_type = "week",
  geo_values = "*",
  time_values = epirange(201552, 202901)
)
```

Hint: The `epiprocess` documentation and vignettes are available at cmu-delphi.github.io/epiprocess.

- **Check out your dataset! Use `view()` first, for a quick look. Then use `summary()` to look at the dataset's metadata and overall statistics. For information on what each section means, visit https://cmu-delphi.github.io/epiprocess/dev/reference/print.epi_df.html**

ⓘ Look at the "Latency" section in the `summary()` output. The (!) symbol appears when there is a "notable" lag. For an up-to-date indicator, you would see no warning and typically a lag of only a few days. If your chosen indicator has a notable lag, how does that impact its use for real-time surveillance?

- **Use `epi_slide_mean()` and `epi_slide_sum()` to calculate moving averages and weekly totals in a new column of your `epi_df`.**
  - Try `.window_size` and `.align` parameters to control the sliding window size and the alignment of the sliding window, respectively.

Hint: Because this data has `time_type = "week"`, `.window_size` must be expressed as a `difftime` (e.g., `as.difftime(3, units = "weeks")`).

ⓘ Plot the different columns you calculated, and compare the results. How does changing the `.window_size` from 1 to 3 weeks impact the "noise" in your plot? Is there a trade-off between smoothness and time-accuracy?

- Use `autoplot` and `plot_heatmap` to explore the spatial and temporal trends in the data.
  - `autoplot`
    - Try `.facet_by = "geo_value"` to see each location in its own box.
    - Set `.interactive = TRUE` to enable zoom, hovering, and data tooltips.
  - `plot_heatmap`
    - Change the x-axis range using `coord_cartesian()`

ⓘ If not included already, try selecting all states and use `plot_heatmap()` to identify which states show the most synchronous highs or lows of the metric you're viewing. Can you spot any regional clusters or outliers? Do you see a pattern in where data is most frequently missing?

- Estimate the indicator's relative change using `growth_rate()` and plot the original metric and the growth rate side by side:
  - Filter your data to a U.S. state of interest (ex. "mi") and remove all "NA" values from the column `value`.
  - Arrange your data by the column `time_value` and then calculate the growth rate of `value` in a new column using `growth_rate()`
  - Use ggplot to plot both the column `value` and your growth rate column together in one line plot – Notice anything odd?
    - Look at the scale of `value` vs. the scale of your growth rate column - Make adjustments to your plot as necessary to view both values.

ⓘ What does the growth rate measure? How is it changing as your data changes?

⚔️ Explore rtestim to estimate the effective reproductive number, Rt, using a case count indicator.

2.2 Compare two different indicators

⭐ **Explore other available indicators with the Delphi EpiPortal (https://delphi.cmu.edu/epiportal/), or, follow along with these steps using the NCHS Influenza Deaths (Weekly new, per 100k people) and ILINet's weighted percent influenza-like illness rates for Pennsylvania**

```
epidata_nchs_mortality_state <- pub_covidcast(
  source = "nchs-mortality",
  signals = "deaths_flu_incidence_prop",
  geo_type = "state",
  time_type = "week",
  geo_values = "pa",
  time_values = epirange(201952, 202901)
)


epidata_fluview <- pub_fluview(
  regions = "PA",
  epiweeks = epirange(201952, 202901)
)
```

- Join your two datasets into a single WIDE FORMAT `epi_df`. [Hint: turn the two sets into individual `epi_df`'s first, then join them]
- Use `autoplot()` with both indicators selected as parameters to view the data on one plot.
- Calculate a correlation using `epiprocess::epi_cor()`. [Hint: You can find the "correlation" vignettes [here](#)]
    - Set the `cor_by` parameter to `geo_value` and `time_value` to estimate the correlation over time within the same location (per-location temporal correlation) and across regions at a specific moment (correlation over time).
    - You can also vary the `method` (using `"pearson"` for linear relationship or `"spearman"` for ranked-based correlation).
- Use `ggplot2` or any other plotting package to visualize how the correlation changes over time.

ⓘ If you see a high spatial correlation but a low temporal correlation, what does that tell you about the relationship between the two indicators? Is it possible for two signals to be highly correlated in space but not in time?

⚔️ Use the `dt1` and `dt2` parameters in `epi_cor()` to find the exact lag that maximizes the correlation between your candidate and guiding indicators. You can even wrap this in a loop or `lapply()` over a range of lags to build a "lag-lead" visualization of the relationship between the two signals.

2.3 Revision behavior - Understanding how data matures over time

For version-aware processing, including revision analysis and modeling, the training data needs to include the revision history. Since we're using Delphi-hosted data right now, we can fetch data versions from the Delphi Epidata API by specifying `issues = "*"` in the API call.

⭐ **Fetch and visualize the version history of an indicator - you can choose your own indicator, or follow along with the Day-adjusted COVID-related Doctor Visits indicator.**

```
archive_res <- pub_covidcast(source = "doctor-visits",
                             signals = "smoothed_adj_cli",
                             geo_type = "state",
                             time_type = "day",
                             geo_values = c("mi", "ny", "tx", "pa"),
                             time_values = epirange(20220101, 20220301),
                             issues = "*")
```

- Convert the data into the `epi_archive` format using `as_epi_archive()`. [Hint: `as_epi_archive()` requires a column called `version.` Since the Delphi Epidata API returns the `issue` column, you may need to rename this column with `rename(version = issue)`]
- Plot the data using `autoplot()`. [Hint: To view the help file for this method, type `?autoplot.epi_archive` in R or visit https://cmu-delphi.github.io/epiprocess/reference/autoplot-epi.html]

ⓘ Look at the divergence in the `autoplot()` lines. Which version tends to be the most "optimistic" or "pessimistic" compared to the final data (in black)? Do you notice any systematic bias in the initial reports?

- “Slice” the archive at a specific version date using `epix_as_of()`. [Documentation: https://cmu-delphi.github.io/epiprocess/dev/reference/epix_as_of.html]

```
one_slice <- epix_as_of(archive_res, version = as_date("2022-02-01"))
```

ⓘ When you extract a snapshot using `epix_as_of()`, you are essentially "time-travelling" to see the state of knowledge on that date. If the preliminary values on a date were much lower than the final values, how might that lead to a false sense of security during an emerging surge?
ⓘ How might this ability to view history and current in a report assist in decision-making around a particular metric? Do you handle this in any particular way at your institution/agency?

⚔️ Instead of fetching the entire history, use the `as_of` parameter in `pub_covidcast()` to fetch the data exactly as it was reported on a specific date. Convert it to an `epi_df` and compare it to the slice you took above.

- Use `revision_analysis()` [https://cmu-delphi.github.io/epiprocess/dev/reference/revision_analysis.html] to explore how signals change across revisions.

```
revision_analysis(archive_res, all_of('value'))
```

Hint: use `$revision_behavior` on the object returned by `revision_analysis()` to access the summary statistics dataframe for each revision date (i.e. “key”).

ⓘ A key metric generated by `revision_analysis()` is `lag_near_latest` (often called stabilization lag), which measures the number of days it takes for a data point to “settle” within a reliable threshold (e.g., 20%) of its final backfilled value.

If the average stabilization lag is 20 days, what does that imply for a public health official trying to use yesterday’s data to make a decision? How might you adjust your confidence in a signal based on these maturity metrics?

⚔️ Try out using the `within_latest` parameter within `revision_analysis()` to change the reliable threshold value. How does changing that value affect the output value of `lag_near_latest`?

⚔️ Use dplyr’s `group_by()` and `summarize()` to calculate revision statistics for each location.

Interested in trying this sort of analysis on your own data? To turn a dataset of your own into an `epi_df`, it only needs the `geo_value` and `time_value` columns. For more information on this, visit: https://cmu-delphi.github.io/epiprocess/dev/articles/epi_df.html

---

Example solutions!